Practical-3

Aim:

To Implementation of max-heap sort algorithm

Algorithm:

1. Bubble sort
Algorithm:
1. Start with the first element
2. Compare it with the next element
3. If the first element is greater, swap them
4. Continue comparing adjacent elements until the end
5. Repeat the process for all elements
6. Stop when array is sorted

```
for(i=0;i<n-1; i++){
    for(j=0 ; j<n-i-1 ;j++){
        if (arr[j] > arr[j+1]){
            swap(arr[j], arr[j+1]);
        }
    }
}
```

1. Selection Sort:
Algorithm:
1. Start from the first element of the array
2. Assume the first element is the minimum element
3. Compare it with remaining all elements.
4. Find the smallest element
5. Swap the smallest element with the first element
6. Move to the next position
7. Repeat the same process until the array is sorted
8. Stop when all elements are in correct positions.

```
for(int i=0;i<n-1;i++){
    int minIndex=i;
    for(int j=i+1; j<n; j++){
        if(arr[j] < arr[minIndex]){
            minIndex=j;
        }
    }
    int temp= arr[i];
    arr[i] = arr[minIndex];
    arr[minIndex]= temp;
}
```

Insertion Sort:
1. Start from the second element
2. Store it in key
3. Compare key with the elements before it
4. Shift larger elements one position to the right
5. Insert key in its correct position
6. Repeat until the array is sorted

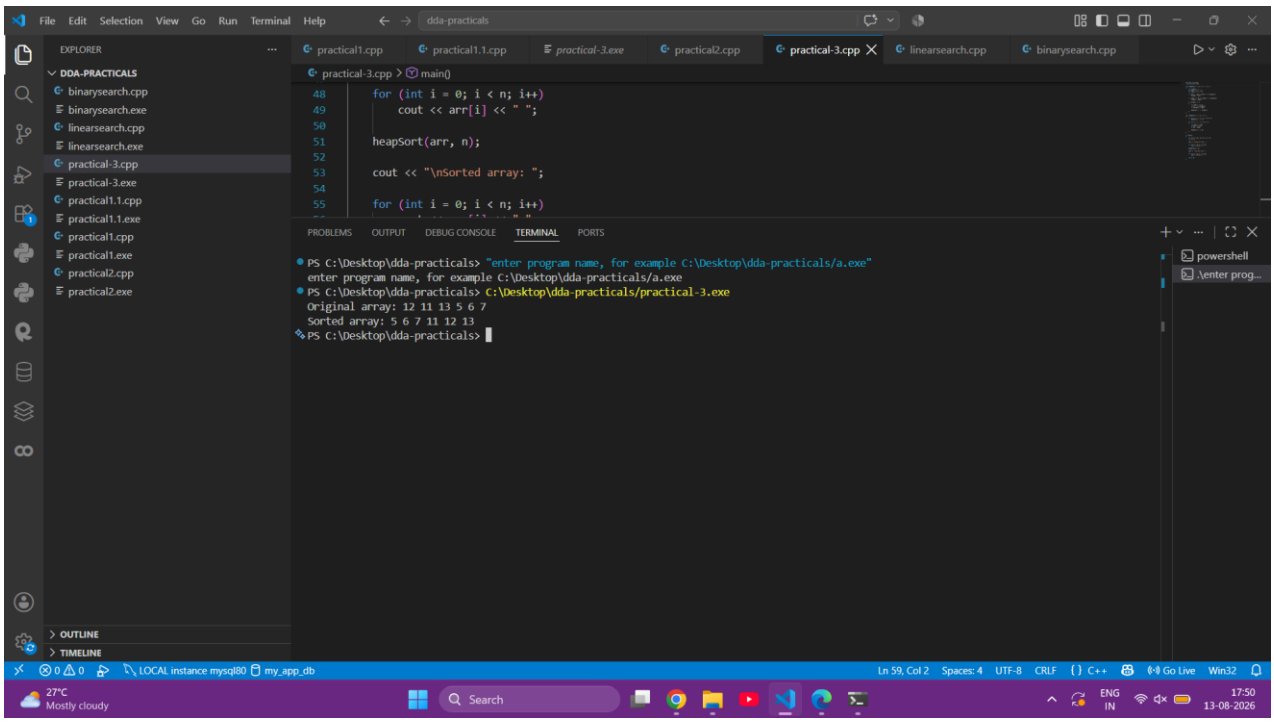
```
for(int i=1;i<n;i++){
    int key = arr[i];
    int j = i-1;
    while(j>=0 && arr[j]>key){
        arr[j+1]= arr[j];
        j--;
    }
    arr[j+1]= key;
}
```

Code:

```
#include <iostream>
using namespace std;

void heapify(int arr[], int n, int i)
{
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i)
    {
        int temp = arr[i];
        arr[i] = arr[largest];
        arr[largest] = temp;

        heapify(arr, n, largest);
    }
}

void heapSort(int arr[], int n)
{
    // Build Max Heap
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    // Heap Sort
    for (int i = n - 1; i > 0; i--)
    {
        int temp = arr[0];
        arr[0] = arr[i];
        arr[i] = temp;

        heapify(arr, i, 0);
```

```
    }
}

int main()
{
    int arr[] = {12, 11, 13, 5, 6, 7};
    int n = 6;

    cout << "Original array: ";

    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";

    heapSort(arr, n);

    cout << "\nSorted array: ";

    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
 return 0;
}
```

Output:




Marwadi
University
Marwadi Chandarana Group
NAAC
A+

Result:

The Max-Heap Sort algorithm was successfully implemented and executed. The given elements were sorted in ascending order, and the time complexity was analyzed for the best, average, and worst cases. Thus, the implementation of the **Max-Heap Sort algorithm** was successfully completed.


Marwadi
University
Marwadi Chandarana Group
NAAC
A+